

Report VI

Fitting tabulated data using singular values decomposition (SVD)

Author: Patryk Kuna

1. Introduction

In many numerical problems we are given a discrete set of data points representing an unknown function and the goal is to approximate it with a simpler analytical form. A common approach is polynomial fitting.

Given a dataset

$$\{(x_0, y_0), (x_1, y_1), \dots, (x_{m-1}, y_{m-1})\}$$

we approximate it with a polynomial of degree $n - 1$:

$$f(x) = \sum_{k=0}^{n-1} c_k x^k$$

Substituting all nodes x_i leads to a system of linear equations:

$$A\vec{c} = \vec{y}$$

where matrix A is constructed from powers of x . For $m > n$, the system is overdetermined and cannot be solved exactly.

To obtain the optimal solution in the least-squares sense, the **Singular Value Decomposition** is used. It decomposes matrix A into:

$$A = U\Sigma V^T$$

and allows stable computation of coefficients:

$$\vec{c} = V\Sigma^{-1}U^T\vec{y}$$

This method is numerically robust and particularly useful when the linear system is ill-conditioned (small changes dramatically shift the solution).

2. Description of the Numerical Task

The task consisted of fitting the function:

$$y(x) = x \cdot \cos(x)$$

in the interval:

$$x \in [0, 3\pi]$$

The following steps were performed:

- generation of m evenly spaced nodes
- computation of corresponding values y_i
- construction of matrix A using powers of x
- performing SVD decomposition
- reconstruction of matrix A
- computation of polynomial coefficients
- visualization of results

Three cases were considered:

- $m = 8, n = 6$
- $m = 30, n = 10$
- $m = 30, n = 20$

3. Numerical Method

The solution is based on Singular Value Decomposition:

$$A = U\Sigma V^T$$

The coefficients are computed using:

$$\vec{c} = \sum_{k=0}^{n-1} \frac{1}{\sigma_k} \vec{v}_k (\vec{u}_k^T \vec{y})$$

This formulation shows that:

- data is projected onto left singular vectors
- contributions are scaled by inverse singular values
- final coefficients are constructed from right singular vectors

Additionally, matrix reconstruction is performed:

$$A = \sum_{k=0}^{n-1} \sigma_k \vec{u}_k \vec{v}_k^T$$

which serves as a verification of correctness.

4. Implementation

The program was written in C++ it uses the **LAPACK** library for SVD computation.

Main steps of implementation:

1. Data generation
 - uniform distribution of nodes
 - computation of $y(x) = x \cdot \cos x$
2. Matrix construction
 - $A_{i,j} = x_i^j$
3. SVD computation
 - Using routine LAPACK_dgesvd()
4. Matrix reconstruction
 - verifying correctness of decomposition
5. Coefficient computation
 - implementation of formula using singular vectors

6. Saving the results

- singular values
- coefficients
- reconstructed matrix
- data for plotting (100 points)

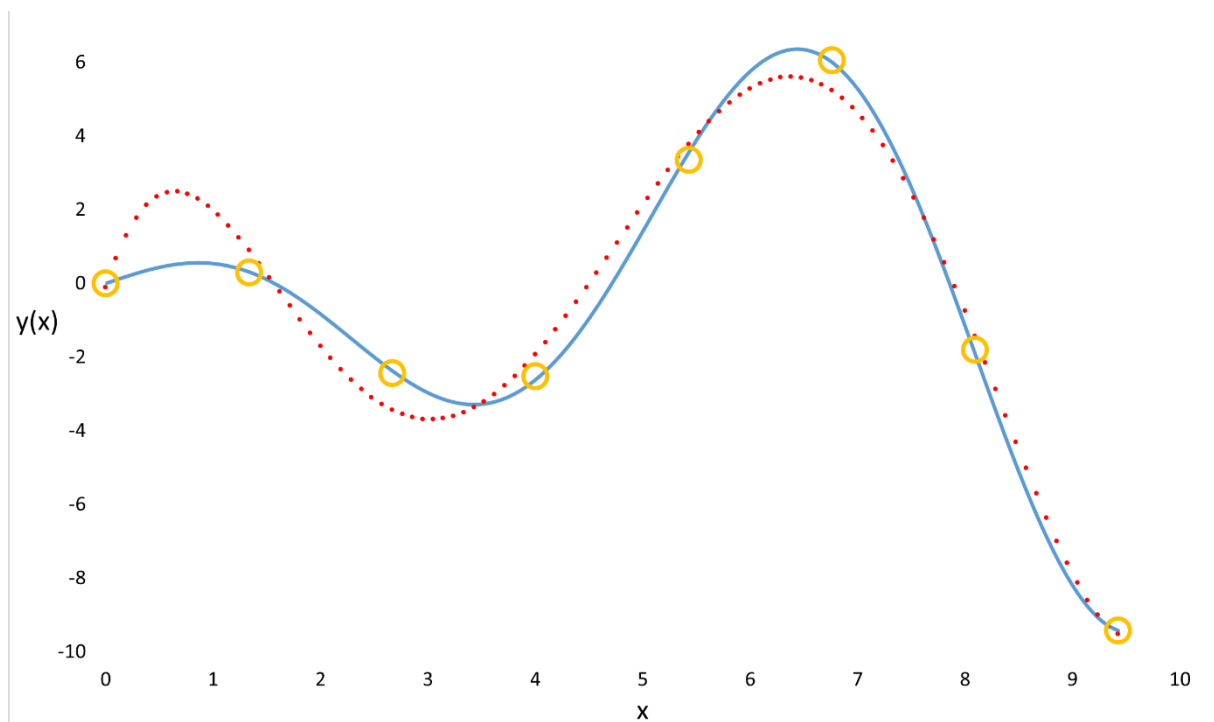
5. Results

The plots below show the input data and the fitted polynomial for three cases.

Case 1: $m = 8, n = 6$

- the polynomial captures the general trend
- visible deviations occur in regions with high curvature
- limited number of nodes reduces accuracy

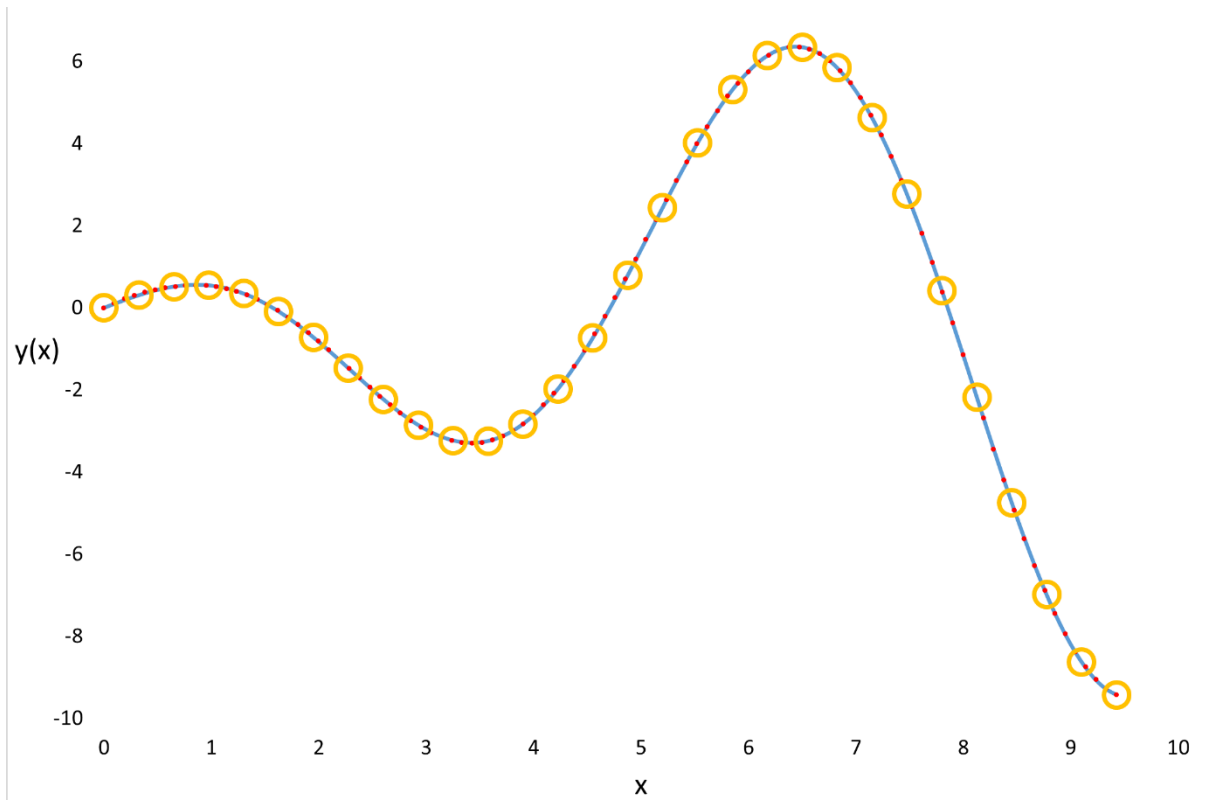
Figure 1. Input data (yellow circles), exact function $y(x) = x \cdot \cos(x)$ (red dotted line), and polynomial fit (blue solid line) for $(m, n) = (8, 6)$.



Case 2: $m = 30, n = 10$

- significantly better agreement
- almost perfect overlap with exact function $y(x)$
- reduced error across the interval

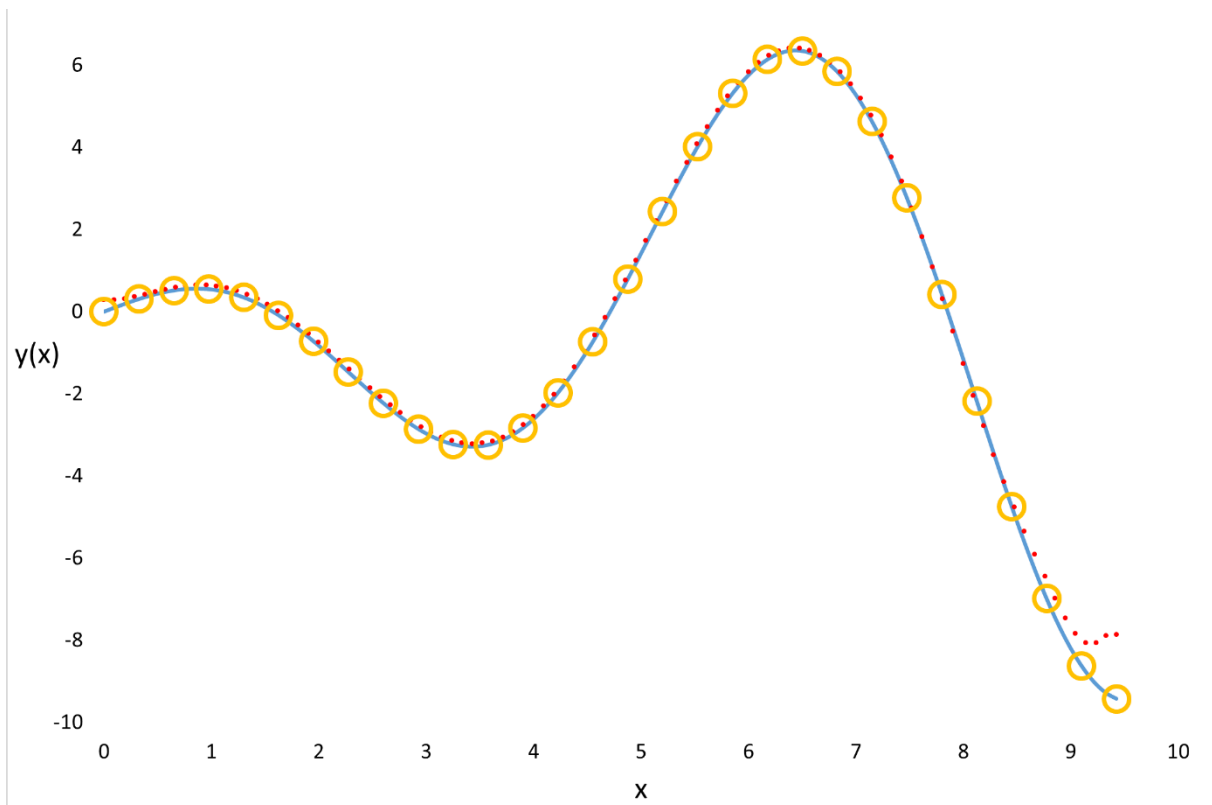
Figure 2. Input data (yellow circles), exact function $y(x) = x \cdot \cos(x)$ (red dotted line), and polynomial fit (blue solid line) for $(m, n) = (30, 10)$.



Case 3: $m = 30, n = 20$

- quite accurate fit
- overlap with the function is great until $x = 8$
- oscillations appear near the boundaries of the interval

Figure 3. Input data (yellow circles), exact function $y(x) = x \cdot \cos(x)$ (red dotted line), and polynomial fit (blue solid line) for $(m, n) = (30, 20)$.



Higher polynomial degree increases flexibility but may introduce overfitting effects.

Singular Values

- singular values decrease rapidly
- smaller values correspond to less significant components
- very small values may amplify numerical errors

Matrix Reconstruction

- reconstructed matrix matches original matrix
- differences are negligible
- confirms correctness of SVD implementation

6. Conclusions

The SVD method provides a stable and effective way to solve overdetermined systems arising in polynomial fitting.

The following observations were made:

- increasing number of data points improves the accuracy
- higher polynomial degree leads to better approximation but may cause oscillations, especially near the boundaries of an interval
- singular values determine importance of components in the solution
- matrix reconstruction confirms correctness of decomposition
- SVD ensures numerical stability

Overall, the method proved to be reliable and well-suited for fitting tabulated data.